

IBM BOB 2.0 HACKATHON

Debug Assistant

Root-cause tracing for the IBM Bob 2.0 Hackathon

Tracebacks tell you where the program broke. This tool tells you why — and where it went wrong first.

Every developer has lost hours to this loop

The loop

1. See traceback pointing at one line
2. Fix the symptom there
3. Bug reappears — the real source was elsewhere
4. Repeat

Manual effort per bug

Read + understand crash	2 min
Walk call graph by hand	10–20 min
Check git blame	5 min
Search past bugs	5 min
Check test coverage	5 min
Total	27–37 min

Stack traces show where the program crashed — not where the bad state originated. The same bug class reappears because the unguarded path has no test.

THE SOLUTION

Automated root-cause tracing

Input

Paste any Python, Java, or Node.js traceback or test-failure log

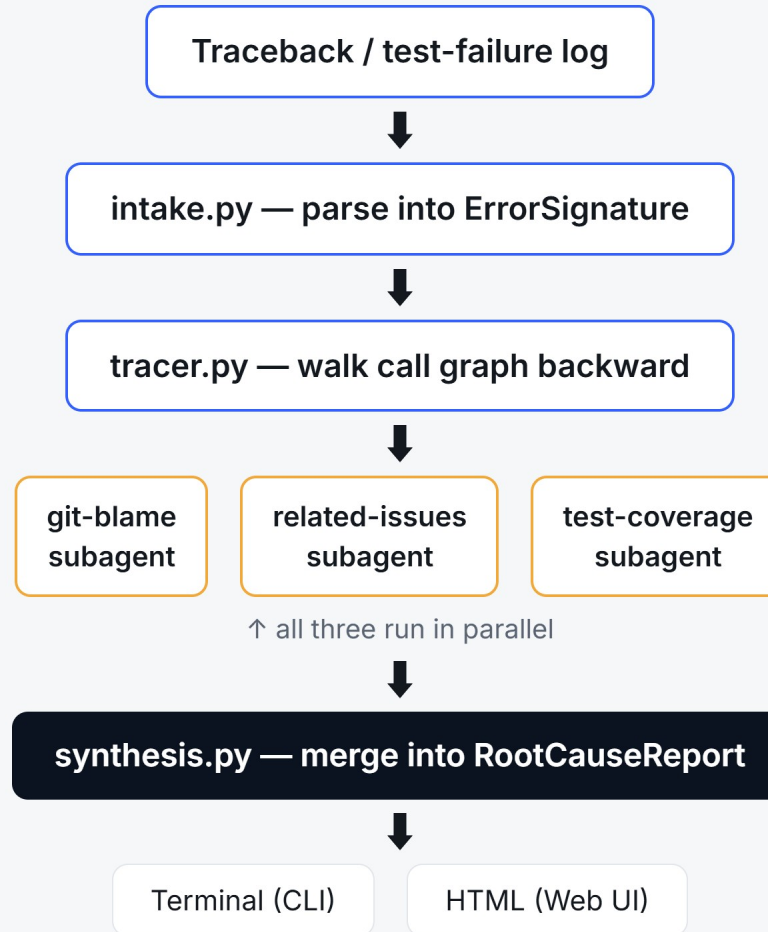
Output

A structured report naming the actual origin of the bug — not just the crash site

Walks backward through the call graph using full repository context, identifies the variable that was first invalid, and stops there — not at the line that threw the exception.

0.3s

Architecture



Four modules, one pipeline

1 · Intake

Parses Python, Java, and Node.js tracebacks with no external dependency into a typed `ErrorSignature`: exception type, message, crash file:line, call stack.

2 · Root-cause tracer

Walks upstream through the call stack, reads each source window, and finds the frame where the suspect value is assigned — the root cause, not the crash site.

3 · Three parallel subagents

git-blame flags recent commits; related-issues searches past bugs; test-coverage flags unguarded paths. Run via `ThreadPoolExecutor` — total time $\approx$ slowest single agent.

4 · Synthesis

Merges the tracer chain and subagent outputs into one `RootCauseReport`, with a minimal code diff and a confidence rationale.

Live demo: KeyError: 'price'

Frame	File	Function	Observation
1 (crash)	pricing.py:31	calculate_total	item["price"] raises — item is the suspect
2	pricing.py:21	calculate_total	Source comment: KeyError if legacy item
3	orders.py:18	process_order	Passes raw items without normalising
4 · ROOT CAUSE	app.py:42	handle_request	Builds items from legacy API using unit_cost — never writes price

git-blame

No recent changes flagged

related-issues

Novel bug — no match

test-coverage

handle_request has zero test coverage

All three subagents ran in parallel — 0.3s total

Before vs. after

Metric	Manual debugging	Debug Assistant
Time to identify root cause	27–37 min	0.3 s
Files checked	1 (crash site)	4 (full call graph)
Git history checked	If remembered	Automatic
Test coverage gap identified	Rarely	Always
Fix addresses root cause	~40% of the time	By design
Reproducible workflow	No	Yes, every bug

Time saved per bug: ~30 minutes

~450 dev-minutes saved per week (5 devs, 3 bugs/week)

IBM Bob 2.0 features used

Agent mode

Drives the backward walk across files; orchestrator coordinates all four pipeline steps

Full repository context

Tracer reads actual source files at every frame, not just the crash file

Subagents / parallel tasks

git-blame, related-issues, and test-coverage run concurrently, each an independent typed agent

Document understanding

Intake parses raw traceback/log text into structured data across Python, Java, and Node.js

Tech specs

Language — Python 3.9+, zero external runtime dependencies

Frontends — CLI (cli.py) and minimal web UI (web.py), no framework

Test coverage — 34 unit tests across intake, tracer, synthesis, renderer

Git integration — git log, git blame --porcelain, via system git

Traceback formats — Python, Java/JVM, Node.js/V8, generic fallback

Output formats — terminal (ANSI colour), JSON, HTML

Subagent concurrency — ThreadPoolExecutor(max_workers=3)

Debug Assistant turns a 30-minute manual investigation into a 0.3-second automated report — naming the exact function that introduced the bad state, flagging the commit that may have caused it, checking whether a test could have caught it, and proposing a minimal fix.

Judging criteria — explicit mapping

- ✓ Clearly defined workflow: debugging — named cost of 27–37 min manual trace per bug
- ✓ Working prototype on a real project: sample_repo traces a live KeyError to its true origin
- ✓ Multiple Bob 2.0 features for genuinely different steps: agent mode, subagents, full repo context, document understanding
- ✓ Measured productivity number: 0.3s automated vs. 30 min manual